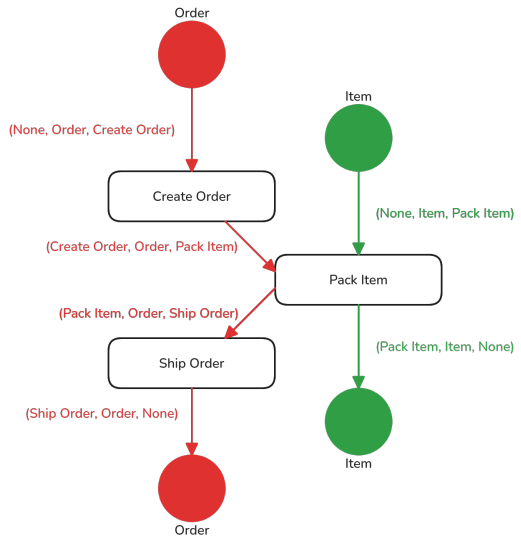
Plugin Evaluation

In this evaluation, you will integrate a process mining implementation into Ocelescope by creating a plugin.
The goal is to create a new plugin using the existing Ocelescope system and its documentation.

Let's say you have already written two Python functions:

1. A **discovery function** (`discover_dfg`) that discovers an *object-centric directly-follows graph (OC-DFG)* from an Object-Centric Event Log (OCEL).
   It returns a list of tuples in the form `(activity_1, object_type, activity_2)` , where each tuple means that `activity_2` directly follows `activity_1` , for the given `object_type` .
   Start and end activities use `None` to indicate the absence of a preceding or following activity.

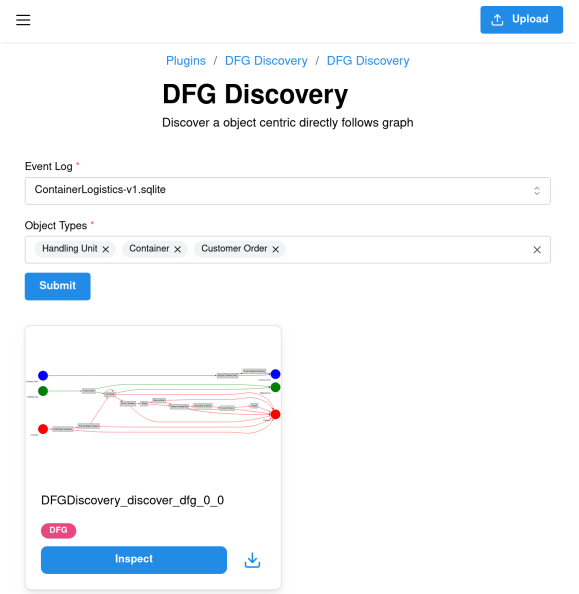
   Example Output of discover_dfg

   Visualization of a possible output from `discover_dfg` :
   [(None, Order, Create Order), (Create Order, Order, Pack Item), (Pack Item, Order, Ship Order), (Ship Order, Order, None), (None, Item, Pack Item), (Pack Item, Item, None)].

2. A **visualization function** (`convert_dfg_to_graphviz`) that creates and returns a Graphviz `Digraph` instance representing the DFG, which can later be used to generate images.

By the end of this evaluation, you will have a **working plugin** that looks like this:



Example of a completed OC-DFG discovery plugin in Ocelescope.

For additional context or examples, you can use the **Plugin Development Guide** and the **Tutorial**.
Everything you need to complete this evaluation is included here, but if you're curious and want to explore the topic further, those guides provide a deeper look into plugin development in Ocelescope.

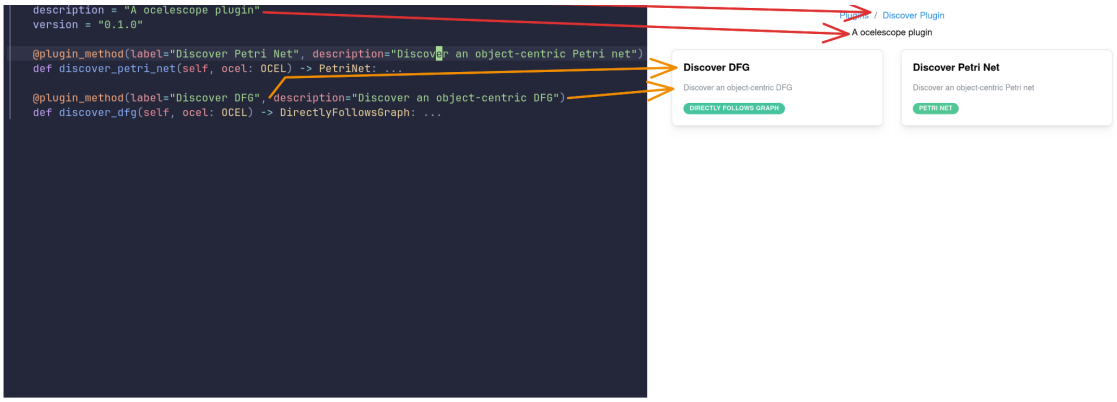
Step 1: Crash course in Ocelescope

Before we start building our plugin, let's take a quick look at the main building blocks of an Ocelescope plugin. Understanding these core concepts will make it easier to follow the next implementation steps.

Plugin Class

An **Ocelescope plugin** is a collection of Python functions grouped inside a class that inherits from the base `Plugin` class provided by the `ocelescope` package.

Each plugin includes basic **metadata**, such as its name, version, and description, defined as class variables. Each function inside a plugin class that is decorated with `@plugin_method` becomes a callable action in the Ocelescope interface.



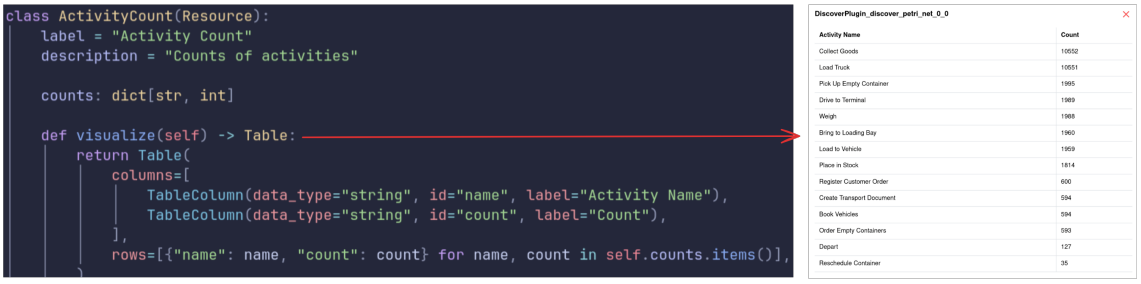
Example of an Ocelescope plugin in code and in the app.

Resources

Resources are Python classes that can be used as inputs and outputs of plugin methods. They can represent process models, results of performance analyses, or any other structured data.

Resources returned by plugin methods are automatically saved and can be reused as inputs for other methods. A resource is a Python class that inherits from `ocelescope.Resource`.

A resource can optionally implement a visualization function, which returns one of Ocelescope's built-in visualization types. This allows the resource to be displayed automatically in the frontend.



A resource used to store activity counts. On the left its Python implementation; on the right, its visualization in Ocelescope.

Plugin Methods

As discussed earlier, plugin methods are functions defined inside a plugin class. Their input parameters automatically generate a corresponding form in the Ocelescope frontend.

A plugin method can have any number of parameters of type `OCEL` or `Resource`. In addition, it

can include one plugin input parameter, defined by creating a custom class that inherits from the `PluginInput` base class provided by the `ocelescope` package.
This custom class, called a *configuration input*, defines the user-configurable parameters for the plugin method.

Example: Defining a Plugin Method with an OCEL and a Custom Input Class

The following example shows how a plugin method can include both an `OCEL` parameter and a custom input class that inherits from `PluginInput`. The input class adds extra configurable parameters, in this case a numeric `frequency` field.

```python
from ocelescope import OCEL, Plugin, PluginInput, plugin_method

class FrequencyInput(PluginInput):
    """Defines the input parameter for the frequency filter plugin."""
    frequency: int

class FrequencyFilterPlugin(Plugin):
    ...

    @plugin_method(
        label="Filter Infrequent Object Types",
        description="Removes object types that occur less than the given
frequency threshold."
    )
    def filter_infrequent(
        self,
        ocel: OCEL,
        input: FrequencyInput,
    ) -> OCEL:...
```

You can also define special *OCEL-dependent fields* within the same class using the `OCEL_FIELD` helper.
This helper links a field to the `OCEL` parameter of a plugin method, allowing you to create input fields that are automatically populated with elements such as object types, activities, or attribute names from the referenced OCEL log.

Example: Using OCEL-dependent fields

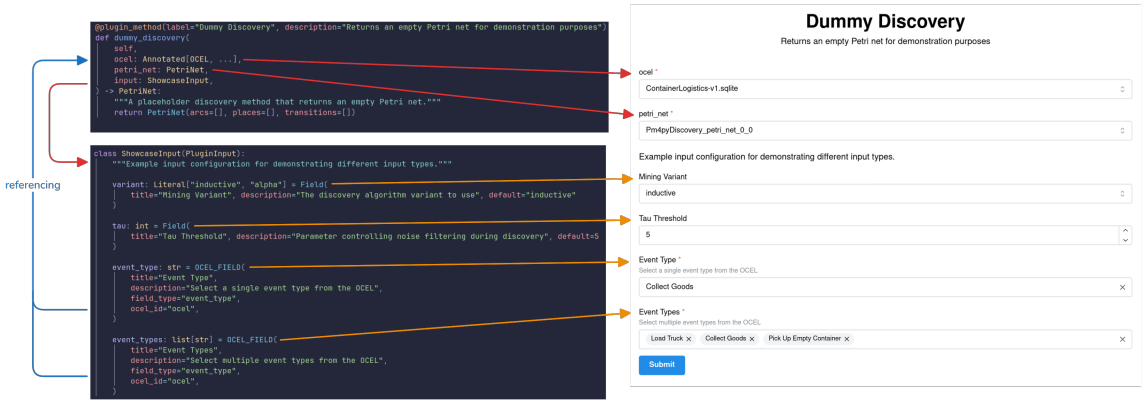
```python
from ocelescope import OCEL, Plugin, PluginInput, plugin_method,
OCEL_FIELD

class Input(PluginInput):
    event_types: list[str] = OCEL_FIELD(
        ocel_id="ocel",
        field_type="event_type",
    )

class EventTypeFilter(Plugin):
    label = "Event Type Filter"
    description = "A plugin that filters out selected event types"
    version = "0.1.0"

    @plugin_method(label="Filter Events", description="Filters events by
type")
    def filter_out_events(
        self,
        ocel: OCEL,
        input: Input,
    ) -> OCEL:
        ...
```

This example shows how to define OCEL-dependent fields using the `OCEL_FIELD` helper, which links a field (here, `event_types`) to the `OCEL` parameter of the plugin method. The `ocel_id` value must match the name of the corresponding `OCEL` parameter.



A plugin method with its custom input class. On the left is the Python code, and on the right is the automatically generated form in Ocelescope.

Step 2: Set Up Your Environment

Let's start by setting up the minimal Ocelescope plugin template.
You can choose one of the following two methods to prepare your project.

Option A - Clone the Template from GitHub

Clone the minimal plugin template directly from Github (link to the repository):

```
git clone https://github.com/Grkmr/minimal-plugin.git
cd minimal-plugin
```

Option B - Generate a New Project with Cookiecutter

Alternatively, you can generate a new plugin project using Cookiecutter through uv:

> **Warning**
>
> When running the Cookiecutter template, always use the default options (press ↵ **Enter** for each prompt). This ensures the generated project matches the structure expected in this evaluation.

```
uvx cookiecutter gh:rwth-pads/ocelescope --directory template
```

When you've completed the setup steps above, your project directory should look like this:

```
minimal-plugin/ <- root
├─ LICENSE
├─ README.md
├─ pyproject.toml
├─ requirements.txt
├─ src/
|  ├─ minimal_plugin/
|  |  ├─ __init__.py
|  |  ├─ plugin.py
```

Install Dependencies

Navigate to the root of the project and install all dependencies using your preferred package manager.

> **Warning**
>
> This evaluation requires **Python 3.13**. Make sure you have it installed before continuing.

Example

```
# With uv
uv sync

# Or with pip
pip install -r requirements.txt
```

Step 3: Implement the Plugin

After setting up the project and becoming familiar with how Ocelescope plugins work, we'll now implement our first real plugin: a discovery plugin for object-centric directly-follows graph (OC-DFGs), as introduced earlier.

The plugin will have the following components:

Inputs

- An **OCEL** log
- A **list of object types** to include in the discovery

Outputs

- A custom **OC-DFG** **Resource** containing the discovered directly-follows graph

Step 3.1 Prepare the Template

The plugin template we set up earlier provides a `plugin.py` file that already includes boilerplate code for a minimal Ocelescope plugin. It contains a plugin class, a resource, and an input class.

Initial state of `plugin.py`

```python
from typing import Annotated

from ocelescope import OCEL, OCELAnnotation, Plugin, PluginInput,
Resource, plugin_method

class MinimalResource(Resource):
    label = "Minimal Resource"
    description = "A minimal resource"

    def visualize(self) -> None:
        pass

class Input(PluginInput):
    pass

class MinimalPlugin(Plugin):
    label = "Minimal Plugin"
    description = "An Ocelescope plugin"
    version = "0.1.0"

    @plugin_method(label="Example Method", description="An example plugin
method")
    def example(
        self,
        ocel: Annotated[OCEL, OCELAnnotation(label="Event Log")],
        input: Input,
    ) -> MinimalResource:
        return MinimalResource()
```

Rename the Resource

1. Rename the class `MinimalResource` to `DFG`.
2. Update the `label` and `description` to indicate that the resource represents an object-centric directly-follows graph.

Rename the Plugin Class

1. Rename the class `MinimalPlugin` to a meaningful name, for example `DiscoverDFG`.
2. Update the `label` and `description` fields to describe the new plugin.
3. Adapt the import in `__init__.py` to reflect the new class name.

Tip

The Ocelescope app looks inside the `__init__.py` file to locate your plugin class.
Make sure to update both the import and the `__all__` list when renaming your plugin.

__init__.py

```python
from .plugin import MinimalPlugin  # Rename this

__all__ = [
    "MinimalPlugin",  # Rename this
]
```

Rename the Plugin Method

1. Rename the method `example` to a descriptive name, for example `discover`.
2. Update the method's `label` and `description` fields to describe its purpose.
3. Adjust the return type hint of the method to use the renamed resource (for example, change `MinimalResource` to `DFG`).

What is a Type Hint?

Type hints are annotations that specify what type of value a function returns or expects as input.
They are written after a function definition using an arrow (`->`).

```python
def discover(...) -> DFG:
    ...
```

Add the Utility File

To keep your plugin code clean and organized, we will place the discovery and visualization functions in a separate file named `util.py`.

You can either **download** the ready-made ⬇ util.py file or **create** a new `util.py` file yourself. In both cases, place it next to your `plugin.py` (i.e. at `src/minimal_plugin/util.py`).

The Discovery and Visualization implementation

You don't need to fully understand the implementation of these functions to complete this evaluation. They are provided as ready-to-use helpers that you will later integrate into your Ocelescope plugin.

util.py

```python
import itertools

import pm4py
from graphviz import Digraph
from ocelescope import OCEL


def discover_dfg(
    ocel: OCEL, used_object_types: list[str]
) -> list[tuple[str | None, str, str | None]]:
    ocel_filtered = pm4py.filter_ocel_object_types(
        ocel.ocel, used_object_types, positive=True
    )
    ocdfg = pm4py.discover_ocdfg(ocel_filtered)
    edges: list[tuple[str | None, str, str | None]] = []
    for object_type, raw_edges in ocdfg["edges"]
["event_couples"].items():
        edges = edges + (
            [(source, object_type, target) for source, target in
raw_edges]
        )

        edges += [
            (activity, object_type, None)
            for object_type, activities in ocdfg["start_activities"]
["events"].items()
            for activity in activities.keys()
        ]

        edges += [
            (None, object_type, activity)
            for object_type, activities in ocdfg["end_activities"]
["events"].items()
            for activity in activities.keys()
        ]
    return edges


def convert_dfg_to_graphviz(dfg: list[tuple[str | None, str, str |
None]]) -> Digraph:
    dot = Digraph("Ugly DFG")
    dot.attr(rankdir="LR")

    outer_nodes = set()
    inner_sources = {}
    inner_sinks = {}
    edges_seen = set()
    types = set()

    for src, x, tgt in dfg:
        if src is not None:
            outer_nodes.add(src)
        if tgt is not None:
            outer_nodes.add(tgt)
        if x is not None:
            types.add(x)
            inner_sources[x] = f"source_{x}"
            inner_sinks[x] = f"sink_{x}"
        edges_seen.add((src, x, tgt))

    # A palette of colors
```

```
    palette = [
        "red",
        "blue",
        "green",
        "orange",
        "purple",
        "brown",
        "gold",
        "pink",
        "cyan",
        "magenta",
    ]
    color_map = {x: c for x, c in zip(sorted(types),
itertools.cycle(palette))}

    # Outer nodes: neutral color
    for n in outer_nodes:
        dot.node(n, shape="rectangle", style="filled",
fillcolor="lightgray")

    # Sources and sinks: colored small circles, with xlabel underneath
    for x in types:
        color = color_map[x]
        dot.node(
            inner_sources[x],
            shape="circle",
            style="filled",
            fillcolor=color,
            width="1",
            height="1",
            fixedsize="true",
            label="",
            xlabel=x,
        )
        dot.node(
            inner_sinks[x],
            shape="circle",
            style="filled",
            fillcolor=color,
            width="1",
            height="1",
            label="",
            fixedsize="true",
            xlabel=x,
        )

    # Rank groups
    with dot.subgraph() as s:
        s.attr(rank="same")
        for n in inner_sources.values():
            s.node(n)

    with dot.subgraph() as s:
        s.attr(rank="same")
        for n in inner_sinks.values():
            s.node(n)

    # Add edges with thicker lines
    for src, x, tgt in edges_seen:
        if x is None:
            continue
```

```
            color = color_map[x]
            if src is not None and tgt is not None:
                dot.edge(src, tgt, color=color, penwidth="2")
            elif src is None and tgt is not None:
                dot.edge(tgt, inner_sinks[x], color=color, penwidth="2")
            elif src is not None and tgt is None:
                dot.edge(inner_sources[x], src, color=color, penwidth="2")

        return dot
```

```
112
113
114
115
116
117
118
119
120
121
122
123
124
```

After completing the previous steps, your project directory should look like this:

```
minimal-plugin/
├─ ...
├─ src/
|  ├─ minimal_plugin/
|  |  ├─ __init__.py
|  |  ├─ plugin.py
|  |  ├─ util.py
```

Solution 1

plugin.py

```python
from typing import Annotated

from ocelescope import OCEL, OCELAnnotation, Plugin, PluginInput,
Resource, plugin_method


class DFG(Resource):
    label = "DFG"
    description = "An object-centric directly follows graph"

    def visualize(self) -> None:
        pass


class Input(PluginInput):
    pass


class DiscoverDFG(Plugin):
    label = "DFG Discovery"
    description = "A plugin for discovering object-centric directly-
follows graphs"
    version = "0.1.0"

    @plugin_method(label="Discover DFG", description="Discovers an object-
centric directly-follows graph")
    def discover(
        self,
        ocel: Annotated[OCEL, OCELAnnotation(label="Event Log")],
        input: Input,
    ) -> DFG:
        return DFG()
```

__init__.py

```python
from .plugin import DiscoverDFG

__all__ = [
    "DiscoverDFG",
]
```

Step 3.2 Integrate the Discovery Functions

Now that the structure is in place, we can integrate the discovery and visualization functions into the plugin to make it functional.

Extend the Resource

Since our plugin returns a directly-follows graph, we should add an `edges` field (class attribute)

to our DFG resource to store the discovered relationships.

The discovery method provided in the `util.py` returns the OC-DFG as a list of tuples `discover_dfg(...) -> list[tuple[str | None , str, str | None]]`. To integrate this into our Resource, extend your `DFG` class to hold this data.

1. Add a field (class attribute) named `edges` with the following type:

```
list[tuple[str | None, str, str | None]]
```

Solution 2

plugin.py

```python
class DFG(Resource):
    label = "DFG"
    description = "An object-centric directly follows graph"

    edges: list[tuple[str | None, str, str | None]]

    def visualize(self) -> None:
        pass
```

Add a visualization to the Resource

Our `DFG` resource can already be used as both an input and an output, but currently it only stores data without any visual representation.

To display it visually in the Ocelescope frontend, we can extend its `visualize` method.

The provided `util.py` file already includes a helper function, `convert_dfg_to_graphviz`, which takes the resource's `edges` as input and returns a `graphviz.Digraph` instance.

Ocelescope supports several visualization types, including `DotVis`, which renders Graphviz DOT strings.

A `DotVis` instance can be created directly from a `graphviz.Digraph` by using `DotVis.from_graphviz(...)`.

Inside the `visualize` method of your `DFG` class:

1. Import the `convert_dfg_to_graphviz` from the `util.py` as a *relative import*.

Use only relative imports

Ocelescope plugins must use **relative imports** when referencing files in the same directory.

```
from minimal_plugin.util import convert_dfg_to_graphviz   # ❌ Do not use absolute imports
from util import convert_dfg_to_graphviz                  # ❌ Do not use top-level imports
from .util import convert_dfg_to_graphviz                 # ✅ Use relative imports instead
```

2. Call the `convert_dfg_to_graphviz` with the resource's `edges` field.
3. Return a `DotVis` instance created with `DotVis.from_graphviz(...)`.

Tip

- You can access the edges through `self.edges`, assuming the field in your `DFG` resource is named `edges`.
- Make sure `DotVis` is imported from the `ocelescope` package before using it:

```
from ocelescope import DotVis
```

Solution 3

plugin.py

```
from ocelescope import OCEL, DotVis, OCELAnnotation, Plugin, PluginInput,
Resource, plugin_method

from .util import convert_dfg_to_graphviz


class DFG(Resource):
    label = "DFG"
    description = "An object-centric directly follows graph"

    edges: list[tuple[str | None, str, str | None]]

    def visualize(self):
        graphviz_instance = convert_dfg_to_graphviz(self.edges)

        return DotVis.from_graphviz(graphviz_instance)
```

Extend the Input Class

Now let's define the input of the `discover` function. Since we renamed the original example method inside the plugin class, it should already include an OCEL parameter named `ocel`.

Because the discovery function allows filtering by object type, we should also allow the user to select which object types to include. This is done by extending the `Input` class.

Inside the `Input` class (which inherits from `PluginInput`):

1. Remove the existing `pass` statement.
2. Add a new field (class attribute) called `object_types` with the type `list[str]`
3. Turn it into an *OCEL-dependent field* using the `OCEL_FIELD` helper, setting the `field_type` to `"object_type"`

Solution 4

plugin.py

```python
from ocelescope import OCEL, OCEL_FIELD, DotVis, OCELAnnotation, Plugin,
PluginInput, Resource, plugin_method

class Input(PluginInput):
    object_types: list[str] = OCEL_FIELD(field_type="object_type",
ocel_id="ocel")
```

Integrate the Implementation

After defining the inputs for our discovery and the `Resource` that will hold the result, we can now connect everything in the `discover` method of our plugin class.

In the `discover` method:

1. Import the `discover_dfg` function as a *relative import*.

Use only relative imports

Ocelescope plugins must use **relative imports** when referencing files in the same directory.

```python
from minimal_plugin.util import discover_dfg  # ❌ Do not use absolute
imports
from util import discover_dfg                 # ❌ Do not use top-level
imports
from .util import discover_dfg                # ✅ Use relative imports
instead
```

2. Call the `discover_dfg` with the `ocel` parameter and the `object_types` field from the

input class, then use the result to create a new instance of the `DFG` resource.

Tip

Always instantiate the `DFG` resource using **named parameters**.

```
return DFG(edges=discovery_result)   # ✅ Correct – use named parameters
return DFG(discovery_result)         # ❌ Incorrect – avoid positional arguments
```

3. Return the created `DFG` resource

Solution 5

plugin.py

```python
from .util import convert_dfg_to_graphviz, discover_dfg

class DiscoverDFG(Plugin):
    label = "DFG Discovery"
    description = "A plugin for discovering object-centric directly-follows graphs"
    version = "0.1.0"

    @plugin_method(label="Discover DFG", description="Discovers an object-centric directly-follows graph")
    def discover(
        self,
        ocel: Annotated[OCEL, OCELAnnotation(label="Event Log")],
        input: Input,
    ) -> DFG:
        edges = discover_dfg(ocel=ocel, used_object_types=input.object_types)

        return DFG(edges=edges)
```

Step 4: Build your plugin

That's it! The final step is to build your plugin. You can do this in one of two ways:

1. **Manually**, by creating a ZIP archive yourself:

```
DfgDiscovery.zip/
├─ plugin/
|  ├─ plugin.py
|  ├─ util.py
|  ├─ __init__.py
```

2. **Using the built-in Ocelescope build command** (recommended):

   Run the build command in the root of your project.

   Make sure to execute it within the same Python environment where you installed your dependencies.

   ```
   ocelescope build
   ```

   Or, depending on how you manage your environment:

   ```
   # If using pipx
   pipx run ocelescope build

   # If using uvx
   uvx ocelescope build

   # If using uv
   uv run ocelescope build
   ```

After building, you'll find your packaged plugin as a `.zip` file inside the `dist/` directory.

Solution 6

Download Plugin

plugin.py

```python
from typing import Annotated

from ocelescope import OCEL, OCEL_FIELD, DotVis, OCELAnnotation, Plugin,
PluginInput, Resource, plugin_method

from .util import convert_dfg_to_graphviz, discover_dfg

class DFG(Resource):
    label = "DFG"
    description = "An object-centric directly follows graph"

    edges: list[tuple[str | None, str, str | None]]

    def visualize(self):
        graphviz_instance = convert_dfg_to_graphviz(self.edges)

        return DotVis.from_graphviz(graphviz_instance)

class Input(PluginInput):
    object_types: list[str] = OCEL_FIELD(field_type="object_type",
ocel_id="ocel")

class DiscoverDFG(Plugin):
    label = "DFG Discovery"
    description = "A plugin for discovering object-centric directly-
follows graphs"
    version = "0.1.0"

    @plugin_method(label="Discover DFG", description="Discovers an object-
centric directly-follows graph")
    def discover(
        self,
        ocel: Annotated[OCEL, OCELAnnotation(label="Event Log")],
        input: Input,
    ) -> DFG:
        edges = discover_dfg(ocel=ocel,
used_object_types=input.object_types)

        return DFG(edges=edges)
```

__init__.py

```python
from .plugin import DiscoverDFG

__all__ = [
    "DiscoverDFG",
]
```

If you'd like, you can upload the ZIP file in the Ocelescope interface to test your plugin directly.

Once finished, return to the evaluation form to complete your assessment.

November 6, 2025